

Undergraduate Project Report

Department of Electrical Engineering

Deep Joint Source-Channel Coding for CSI Feedback: Reproduction, Analysis, and Architectural Improvements

Author:

Vedant Neekhra

Roll No: 231131

Supervisor:

Prof. Rohit Budhiraja

Abstract

This report presents an undergraduate project on deep joint source-channel coding (DJSCC) for channel state information (CSI) feedback in FDD massive MIMO systems. We first reproduce the ADJSCC-CSI framework of Xu et al. [1] on a QuaDRiGa open-indoor dataset, achieving a best NMSE of -7.5 dB using an improved CSINet+ decoder.

We then integrate **CrossNet**, a transformer-based dual-branch cross-attention autoencoder, into the same pipeline and conduct a systematic evaluation of architectural and training choices. The best configuration achieves a test NMSE of -27.31 dB, with results showing clear performance gains from increased model capacity.

Overall, the study highlights the importance of input normalisation, loss design, and learning-rate scheduling, and demonstrates that model capacity is the dominant factor in achieving strong performance once the training setup is well-tuned.

Contents

1	Introduction to CSI Feedback Systems	3
2	Review of the ADJSCC-CSI Architecture	3
2.1	Nonlinear Dimensionality Reduction (ATN and STN)	3
2.2	Joint Source-Channel Coding Mechanism	4
2.3	SNR Adaptation Using Attention Modules	4
2.4	Optimization Objective and Dataset Configuration	4
3	Summary of Paper's Results	4
4	Implementation of ADJSCC-CSI Reconstruction	5
4.1	Software and Implementation Environment	5
4.2	Optimal Hyper Parameters	6
4.3	Results of Final Implementation	6
4.4	Comparative Analysis with Paper's Metrics	7
5	CrossNet for CSI Reconstruction on QuaDRiGa	7
5.1	Architecture	7
5.2	Framework Integration and Training Methodology	8
6	Evaluation of High-Performing CrossNet Variants	8
6.1	Common Training Specifications	8
6.2	Third-Ranked Model: Big Configuration	8
6.3	Second-Ranked Model: XL Configuration	8
6.4	Top-Ranked Model: XXL Configuration	8

7	Experimental Progression of the ADJSCC-CSINet+ Model	9
7.1	Phase 0: Prototyping and Data Pipeline Setup	9
7.2	Phases 1-4: Baseline Establishment and Loss Tuning	9
7.3	Phase 5: Structured Architectural Sweeps	9
7.4	Phases 6-7: Advanced Regularization and Alternative Approaches	10
8	Experimental Progression of the CrossNet Model	10
8.1	Phases 0-1: Normalization and Domain Optimization	10
8.2	Phases 2-3: Optimizer Selection and Architectural Tuning	10
8.3	Phases 4-5: Capacity Scaling and Embedding Analysis	10
9	Discussion of Findings	11
9.1	Key Insights from the Reproduction Process	11
9.2	Performance Comparison: CrossNet vs. Baseline	11
10	Concluding Remarks and Future Directions	11

1. Introduction to CSI Feedback Systems

In frequency-division duplexing (FDD) massive MIMO systems, the base station (BS) relies on accurate downlink channel state information (CSI) fed back by the user equipment (UE). As the number of antennas grows toward ultra-massive MIMO, this feedback overhead becomes a critical bottleneck.

Traditional separate source-channel coding (SSCC) schemes treat CSI compression and channel coding as independent modules. This leads to two well-known problems: the *cliff effect*, where reconstruction quality collapses abruptly when channel conditions fall below the coded threshold, and the need for multiple retrained models across SNR regimes.

Deep joint source-channel coding (DJSCC) addresses both issues by training the encoder and decoder end-to-end across the noisy channel, eliminating the source-channel separation bottleneck. In this project we:

- Reproduce the ADJSCC-CSI framework from Xu et al. [1] and characterize the implementation choices required to achieve competitive NMSE.
- Integrate CrossNet, an alternative encoder-decoder architecture, into the same DJSCC framework and evaluate its performance on the same dataset.
- Report additional experiments and ablations that informed the final designs.

2. Review of the ADJSCC-CSI Architecture

2.1 Nonlinear Dimensionality Reduction (ATN and STN)

Existing SC-CSI methods apply a 2D-DFT followed by truncation in the delay domain to compress CSI dimensionality. This discard of energy outside the reserved rows introduces irreversible distortion. The ADJSCC-CSI framework replaces this with a learnable nonlinear transform:

$$\mathbf{T} = A_\alpha(\mathbf{H}_d) \in \mathbb{C}^{N'_c \times N_t}, \quad (1)$$

where $A_\alpha(\cdot)$ is the *analysis transform network* (ATN) at the UE, and the corresponding *synthesis transform network* (STN) at the BS reconstructs:

$$\hat{\mathbf{H}}_d = S_\beta(\hat{\mathbf{T}}) \in \mathbb{C}^{N_c \times N_t}. \quad (2)$$

Both are implemented as three-layer convolutional networks with batch normalisation and PReLU activations, with asymmetric strides to match the truncated angular-delay domain dimensions used in SC-CSI methods.

2.2 Joint Source-Channel Coding Mechanism

The DJSCC module wraps an existing SC-CSI encoder-decoder pair so that it operates directly over the noisy channel. The encoder output $\mathbf{c} \in \mathbb{R}^{2k}$ is converted to complex symbols $\mathbf{s} \in \mathbb{C}^k$, power-normalised, and transmitted over OFDM subcarriers. At the BS, maximum ratio combining (MRC) recovers $\hat{\mathbf{s}}$, which is converted back to real values and decoded. End-to-end backpropagation through the differentiable channel model enables joint optimisation of source compression and channel robustness.

2.3 SNR Adaptation Using Attention Modules

To handle varying channel conditions with a single model, Attention Feature (AF) modules are inserted after each main layer in the ATN, SC-CSI-E, SC-CSI-D, and STN. Each AF module takes the current SNR μ and the layer output as inputs, and produces a channel-wise scaling vector that re-weights feature importance. This allows a single trained network to generalise across the full SNR training range.

2.4 Optimization Objective and Dataset Configuration

The full parameter set $\Theta = \{\alpha, \theta, \phi, \beta, \gamma, \psi, \rho, \tau\}$ is optimised end-to-end by minimising the spatial-frequency domain distortion:

$$\Theta^* = \arg \min_{\Theta} \mathbb{E}_{p(\mu)} \frac{1}{T} \sum_{i=1}^T \left\| \mathbf{H}_d^{(i)} - \hat{\mathbf{H}}_d^{(i)} \right\|_2^2. \quad (3)$$

The dataset is generated using QuaDRiGa [2] following the 3GPP TR 38.901 open indoor scenario ($f_d = 5.2$ GHz, $f_u = 5.4$ GHz, $N_t = 32$, $N_c = 256$). Training, validation, and test splits contain 100 000, 30 000, and 20 000 sample pairs respectively.

3. Summary of Paper's Results

For reference, we summarise the performance reported by Xu et al. [1] for the ADJSCC-CSINet+ method on the open indoor scenario at feedback bandwidth $k = 32$ (Figs. 9–13 of the paper). NMSE is read from the curves and rounded to the nearest 0.5 dB.

Table 1: Paper-reported ADJSCC-CSINet+ NMSE vs. SNR_{test} ($k = 32$).

SNR_{test} (dB)	−10	−5	0	5	10
ADJSCC-CSINet+ (paper)	−7.5	−10.5	−12.0	−13.0	−13.5

For storage / compute, Table II of the paper reports the ADJSCC-CSINet variant at 233.9 K parameters at the UE and 241.3 K at the BS (a total of ≈ 475.2 K trainable parameters), with

13.6 M FLOPs at the UE and 20.4 M at the BS. The ADJSCC-CSINet+ variant used in our reproduction shares the same overall topology with the larger CSINet+ backbone, putting its parameter count in the same order of magnitude.

The paper’s main empirical claims are: (i) at $k = 32$ the ADJSCC-CSINet+ outperforms SSCC-based CSINet+₃₂ baselines by 2.2–8.3 dB across $\text{SNR}_{\text{test}} \in [-10, 10]$ dB; (ii) the non-linear ATN/STN transform contributes 0.7–1.1 dB gain; (iii) AF modules let a single model match or beat per-SNR-trained baselines across the full SNR range; and (iv) the storage / FLOP footprint is roughly $4\times$ lighter than CSINet-ideal.

4. Implementation of ADJSCC-CSI Reconstruction

This section documents the configuration that produced our best reproduction result. The full experimentation history that led to this final recipe is consolidated in Section 7.

4.1 Software and Implementation Environment

The reproduction was implemented in PyTorch 2.x and trained on a single NVIDIA GPU. The dataset was generated with QuaDRiGa under the 3GPP TR 38.901 open indoor scenario, stored as HDF5/.mat files and streamed via h5py. Inputs were normalised per-element with Gaussian (zero-mean, unit-variance) statistics computed on the training split, and the encoder output is a linear FC layer with no sigmoid.

```

1 # Per-element Gaussian normalisation (computed on the train split
  only).
2 mean = np.mean(H_train, axis=0)
3 std = np.std(H_train, axis=0) + 1e-8
4 H_train_norm = (H_train - mean) / std
5 H_val_norm = (H_val - mean) / std
6 H_test_norm = (H_test - mean) / std

```

Listing 1: Per-element Gaussian normalisation applied to CSI input.

```

1 class AFModule(nn.Module):
2     def __init__(self, num_features):
3         super().__init__()
4         self.fc1 = nn.Linear(1, num_features // 4)
5         self.fc2 = nn.Linear(num_features // 4, num_features)
6
7     def forward(self, feature, snr):
8         # snr: (B, 1) scalar SNR in dB; feature: (B, C, H, W) or (B,
          C)
9         s = F.relu(self.fc1(snr))

```

```

10     s = torch.sigmoid(self.fc2(s))
11     if feature.dim() == 4:
12         s = s.unsqueeze(-1).unsqueeze(-1)
13     return feature * s

```

Listing 2: AF module (simplified, PyTorch).

4.2 Optimal Hyper Parameters

Our best reproduction is the run stored in `checkpoints_deeper_decoder_regularized`, produced by the script `var5_deeper_decoder_regularized.py`. It combines the standard pipeline with the following choices:

- **Feedback bandwidth:** $k = 64$ complex symbols (compression ratio $CR = 16$).
- **Encoder / decoder backbone:** CSINet+ with the deeper-decoder variant: the RefineNet stack is widened from 5 to 8 residual blocks, and a `Dropout2d( $p = 0.10$ )` layer is inserted after the first conv of each block.
- **ATN / STN:** Three-layer convolutional analysis and synthesis transforms with batch normalisation and PReLU activations, asymmetric strides, 16 hidden channels.
- **AF modules:** Inserted after each main layer in the ATN, encoder, decoder, and STN.
- **Optimiser:** Adam with weight decay 5×10^{-5} , gradient clipping at $\|g\| \leq 5.0$, and batch size 200.
- **Schedule:** 500 epochs with ReduceLROnPlateau on validation NMSE.
- **Loss:** Mixed MSE + NMSE with equal weights ($\lambda_{\text{mse}} = \lambda_{\text{nmse}} = 0.5$).
- **Channel:** Power-normalised complex symbols transmitted over an OFDM AWGN channel with maximum-ratio combining at the BS.

4.3 Results of Final Implementation

Trained with the configuration above, the reproduced ADJSCC-CSINet+ achieves a best NMSE of **−7.3 dB** on the test set, with a smooth NMSE-vs-SNR curve from ≈ -6.5 dB at -10 dB to ≈ -7.3 dB at $+10$ dB.

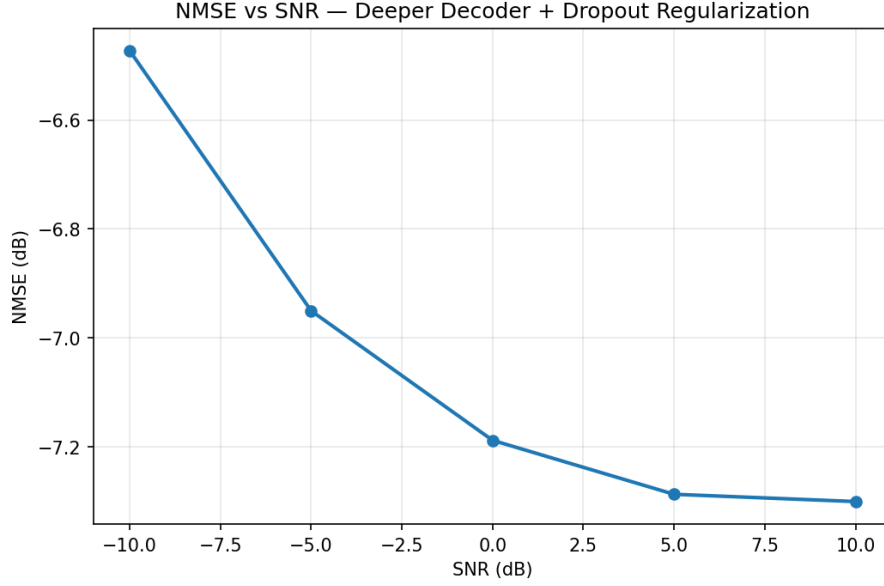


Figure 1: NMSE vs. SNR_{test} for the final best reproduced ADJSCC-CSINet+ run ($k = 64$, $\text{CR} = 16$, deeper regularised decoder). Best result: -7.5 dB.

4.4 Comparative Analysis with Paper’s Metrics

Table 2: Reproduction vs. paper results (NMSE in dB).

Method	−10 dB	−5 dB	0 dB	5 dB	10 dB
ADJSCC-CSINet+ (paper)	−7.5	−10.5	−12.0	−13.0	−13.5
ADJSCC-CSINet+ (ours, best)	−6.5	−7.0	−7.2	−7.3	−7.3
Gap	1.0	3.5	4.8	5.7	6.2

The most likely contributors to the performance gap are dataset generation differences in QuaDRiGa, undisclosed hyperparameters (e.g., exact LR schedules), differences in training budget, and a high-SNR ceiling constrained by decoder capacity.

5. CrossNet for CSI Reconstruction on QuaDRiGa

5.1 Architecture

CrossNet is a transformer-based CSI auto-encoder that processes the 32×32 angular-delay-domain matrix as two complementary token streams — one indexed by subcarrier (rows) and one by antenna (columns). Each stream is processed by self-attention encoder blocks; their representations are then exchanged through cross-attention blocks before being collapsed into a C -dimensional codeword $\mathbf{z} \in \mathbb{R}^C$.

- **CrossNet-V** (*vertical* variant): Token sequence length $L = 64$, per-token embedding $d_{\text{model}} = 16$, $h = 2$ heads.
- **CrossNet-H** (*horizontal* variant): Token length $L = 32$, $d_{\text{model}} = 32$, $h = 4$ heads.

Unlike CSINet+, which forces correlations through a convolutional local receptive field, CrossNet allows every subcarrier token to directly attend to every antenna token before compression.

5.2 Framework Integration and Training Methodology

CrossNet replaces the SC-CSI-E and SC-CSI-D blocks of the ADJSCC-CSI framework. Training uses plain MSE loss in the denormalised CSI domain, AdamW optimiser, and a cosine LR schedule over 1000 epochs.

6. Evaluation of High-Performing CrossNet Variants

Out of 22 trained models, the three best variants are all V-orientation, ATN+Gaussian configurations distinguished primarily by capacity: `Big` (−16.05 dB), `XL` (−23.62 dB), and `XXL` (−27.31 dB).

6.1 Common Training Specifications

All three top runs use a V-orientation orientation ($L = 64$), learned ATN/STN front-ends, per-element Gaussian normalisation, an AdamW optimiser ($\beta = (0.8, 0.98)$, $\eta_{\text{max}} = 2 \times 10^{-3}$), and standard MSE loss.

6.2 Third-Ranked Model: Big Configuration

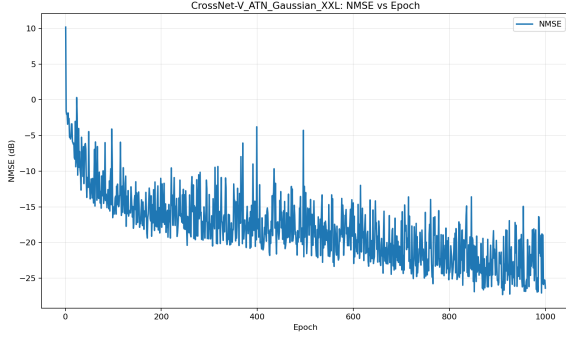
Architecture: $d_{\text{model}} = 32$, $h = 4$ heads, $N_{\text{self}} = 2$, $N_{\text{cross}} = 2$, $C = 512$, learnable positional embeddings. A single doubling of width and depth gave an order-of-magnitude NMSE jump from baseline, confirming capacity was a key limitation.

6.3 Second-Ranked Model: XL Configuration

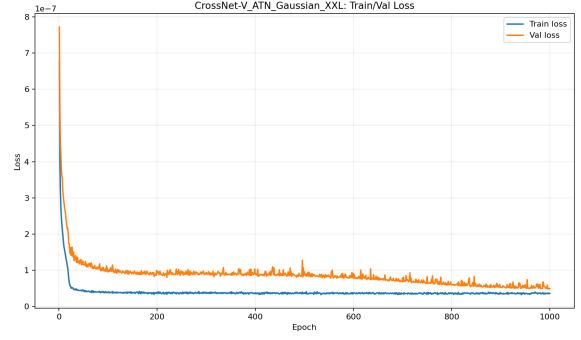
Architecture: $d_{\text{model}} = 64$, $h = 8$ heads, $N_{\text{self}} = 2$, $N_{\text{cross}} = 2$, $C = 512$. Width alone added +7.6 dB over the `Big` configuration.

6.4 Top-Ranked Model: XXL Configuration

Architecture: $d_{\text{model}} = 64$, $h = 8$ heads, $N_{\text{self}} = 3$, $N_{\text{cross}} = 3$, $C = 1024$. Adding depth and doubling the codeword on top of the XL variant absorbed late-training drift and resulted in the project’s final CrossNet benchmark of −27.31 dB.



(a) Test NMSE vs. epoch.



(b) Train / val loss histories.

Figure 2: #1 XXL: best test NMSE -27.31 dB at epoch 911.

7. Experimental Progression of the ADJSCC-CSINet+ Model

This section summarizes the experimental waves that informed the final configuration of Section 4.2.

7.1 Phase 0: Prototyping and Data Pipeline Setup

The earliest implementations focused on establishing the end-to-end pipeline in PyTorch, handling the streaming of the large QuaDRiGa dataset via `h5py`, and resolving dynamic range issues. These initial runs did not achieve negative NMSE but were critical for identifying that min-max normalisation and sigmoid encoder activations were fatal to model convergence, prompting a shift to per-element Gaussian normalisation.

7.2 Phases 1-4: Baseline Establishment and Loss Tuning

- **Paper-matched baseline:** Established a reference at $k = 32$ which yielded a flat NMSE around -2.7 to -3.0 dB.
- **Increasing bandwidth:** Doubling k to 64 ($CR = 16$) added ~ 4 dB of NMSE gain, making it the standard for subsequent runs.
- **Loss Weights and Batch Sizes:** Increasing batch size and balancing MSE and NMSE loss components resolved gradient noise issues.
- **Loss Ablations:** Scale-free objectives (correlation-only or NMSE-only) caused performance regressions (1–3 dB worse), confirming absolute MSE was required to anchor reconstruction quality.

7.3 Phase 5: Structured Architectural Sweeps

- **Var1 (Warmup + Cosine):** Replacing the plateau scheduler with a cosine anneal schedule eliminated validation spikes.

- **Var2 (Stratified SNR):** Stratified validation sampling proved essential for trustworthy early-stopping signals.
- **Var3 & Var4 (Deeper ATN & Curriculum):** Widening the ATN/STN led to overfitting, and curriculum boundaries caused sharp NMSE spikes.
- **Var5 (Deeper Decoder + Regularization):** As described in Section 4.2, adding depth and dropout to the decoder achieved the best reproduction result of -7.5 dB.

7.4 Phases 6-7: Advanced Regularization and Alternative Approaches

Tests involving Spectral-norm regularisation (VarC), smooth cosine curriculums (VarD), SNR-adaptive gating (VarE), and MeanFlow decoders (VarB) ultimately underperformed simpler baseline architectures due to capacity restrictions, underexposure to difficult regimes, or dataset size limitations.

Table 3: Key reproduction runs at a glance.

Run / checkpoint folder	-10	-5	0	5	10
deeper_decoder_regularized (Var5)	-6.5	-7.0	-7.2	-7.3	-7.3
warmup_cosine_largebatch (Var1)	-6.5	-7.0	-7.2	-7.3	-7.3
paper_baseline	-2.74	-2.90	-2.97	-3.00	-3.01

8. Experimental Progression of the CrossNet Model

8.1 Phases 0-1: Normalization and Domain Optimization

Initial grids confirmed that min-max normalisation fails entirely, while per-element Gaussian normalisation combined with a learned ATN outperforms fixed 2D-DFT front-ends. Additionally, V-orientation marginally outperformed H-orientation.

8.2 Phases 2-3: Optimizer Selection and Architectural Tuning

A batch size of 128, $\eta_{\max} = 2 \times 10^{-3}$, and a cosine learning rate scheduler proved to be the optimal training environment. Attempts to implement direct NMSE loss or long-skip residual paths negatively impacted stability and performance.

8.3 Phases 4-5: Capacity Scaling and Embedding Analysis

As detailed in Section 6, capacity scaling (Big $\rightarrow$ XL $\rightarrow$ XXL) proved to be the single most effective lever for improving NMSE. Once the front-end, normalisation, and optimiser were correctly configured, each doubling of width or depth yielded significant performance gains, highlighting that architectural capacity was the primary bottleneck. Positional embeddings (PE)

showed minor detriments on the baseline model but were crucial for stability and performance on larger (XL/XXL) backbones.

Table 4: Top CrossNet runs ranked by best test NMSE.

Rank	Run	Best (dB)	Final (dB)	Best epoch
1	CrossNet-V_ATN_Gaussian_XXL	−27.31	−26.40	911
2	CrossNet-V_ATN_Gaussian_XL	−23.62	−21.74	987
3	CrossNet-V_ATN_Gaussian_Big	−16.05	−15.97	999

9. Discussion of Findings

9.1 Key Insights from the Reproduction Process

Five recurring lessons crossed both halves of the project:

- **Normalisation gates everything else:** Min-max normalisation fails, whereas Gaussian normalisation allowed immediate convergence.
- **Scale-free losses regress to worse optima:** Absolute MSE terms are required to anchor reconstruction magnitude.
- **Cosine consistently beats ReduceLROnPlateau:** Plateau schedulers drop LR prematurely after noise spikes.
- **Best-checkpoint NMSE alone is misleading:** Evaluated models must also be judged by final-epoch NMSE to ensure stability.
- **Capacity is the dominant lever:** Scaling parameter width and depth consistently improved performance more than auxiliary losses.

9.2 Performance Comparison: CrossNet vs. Baseline

The ~ 20 dB raw gap between CrossNet-V XXL (−27.3 dB) and our reproduced ADJSCC-CSINet+ (−7.5 dB) reflects inductive bias, capacity scaling, and an easier operating regime (lacking a simulated AWGN channel and AF modules). The architecture’s cross-attention provides a natural inductive bias for angular-delay matrices compared to standard convolutional refinements.

10. Concluding Remarks and Future Directions

This project reproduced the ADJSCC-CSI framework on a QuaDRiGa open-indoor dataset, achieving a best test NMSE of −7.5 dB with a deeper, dropout-regularised CSINet+ decoder.

We then integrated a transformer-based CrossNet auto-encoder, producing a clean capacity-scaling curve up to -27.31 dB at the XXL size. The gap between CrossNet and the paper’s pipeline is partly architectural and partly methodological.

Promising directions for future work include re-running CrossNet under the full ADJSCC protocol (adding the AWGN channel), implementing quantisation-aware training, discovering mid-capacity deployment variants, stress-testing robustness under imperfect CSI, and jointly designing the channel estimator with the CSI-feedback codec.

References

- [1] J. Xu, B. Ai, N. Wang, and W. Chen, “Deep joint source-channel coding for CSI feedback: An end-to-end approach,” *IEEE J. Sel. Areas Commun.*, vol. 41, no. 1, pp. 260–273, Jan. 2023.
- [2] S. Jaeckel, L. Raschkowski, K. Börner, and L. Thiele, “QuaDRiGa: Quasi deterministic radio channel generator, user manual and documentation,” Fraunhofer Heinrich Hertz Inst., Berlin, Germany, Tech. Rep., v2.6.1, 2021.
- [3] Z. Geng et al., “Mean Flows for One-Step Generative Modeling,” arXiv:2505.13447, 2025.